Airoha IoT SDK for BT Data HID Device Reference Design User Guide

Version: 1.0

Release date: 10 October 2025

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Document Revision History

Revision	Date	Description
1.0	10 October 2025	Initial version.

Table of contents

List of Tables

List of Figures

1 Introduction

The 162x design incorporates two processors: the primary processor is a CM33 and the secondary processor is a RISC-V. The RISC-V processor is specifically designed to handle Human Interface Device (HID) data, while all other processing tasks are managed by the CM33.

The basic architecture can be divided into four main parts: the CM33 application layer, the CM33 service layer, the RISC-V application layer, and the RISC-V service layer. The CM33 handles most of the system's functions, ensuring overall control and coordination, while the RISC-V focuses on specialized HID data processing, optimizing performance for input devices. Each layer is responsible for distinct tasks, which enhances modularity, maintainability, and scalability of the system.

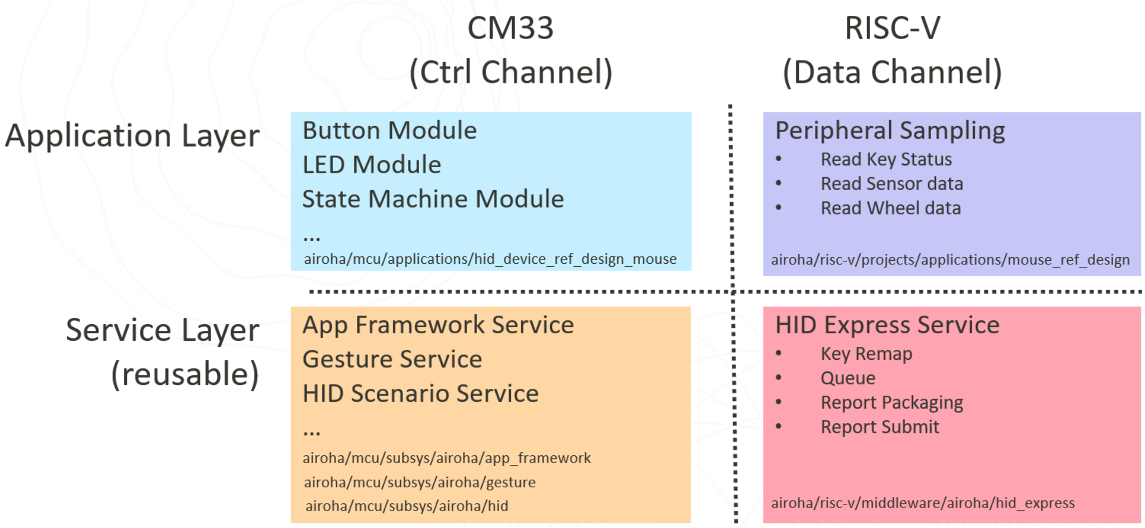

Figure 1. HID Devices Architecture

1.1 EVK Components

Refer to the EVK user guide via `mcu\doc\ab162x\AB162x_Series_EVK_Users_Guide.pdf` for more information about the EVK components.

2 Application

The application is composed of many different modules, with each module responsible for a single, well-defined task. Communication between modules is handled through application framework events, which allow modules to send or receive the latest messages as needed. This architecture significantly reduces direct interactions and dependencies between modules, leading to a higher degree of modularity. As a result, each module can be developed, tested, and maintained independently, making the overall system more flexible and easier to modify or extend in the future. This event-driven approach also enhances code reusability and maintainability, as changes in one module are less likely to impact others.

2.1 Modules

This section introduces several of the most fundamental and important modules.

2.1.1 App State

The App State module is the most important component in the application, responsible for controlling the overall behavior of the app. When an application framework event is received, the App State module makes decisions based on the event details and the current state. For example, upon receiving a mouse button event, the App State module issues an evt_conn_req to initiate a connection request. After the App Scenario module receives this request and completes the necessary processing, it publishes an evt_scenario_status to update the connection status. To simplify the explanation, the app states are divided into three different scenarios. The "connect prepare" state serves as the switching point between these scenarios. Figure 2 to Figure 4 show the state flow of the 2.4G scenario, the standard BT scenario, and the USB scenario.

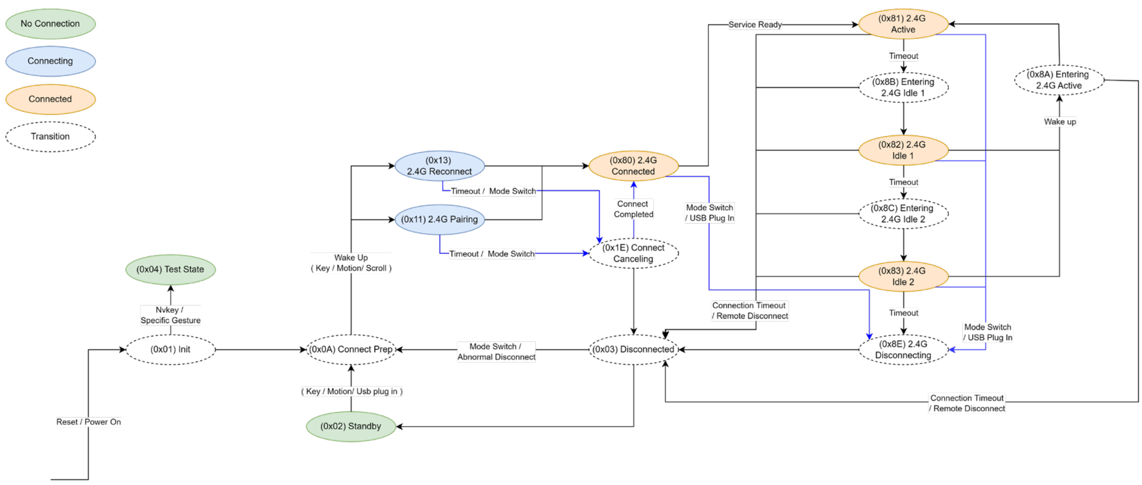

Figure 2. State Flow of 2.4G Scenario

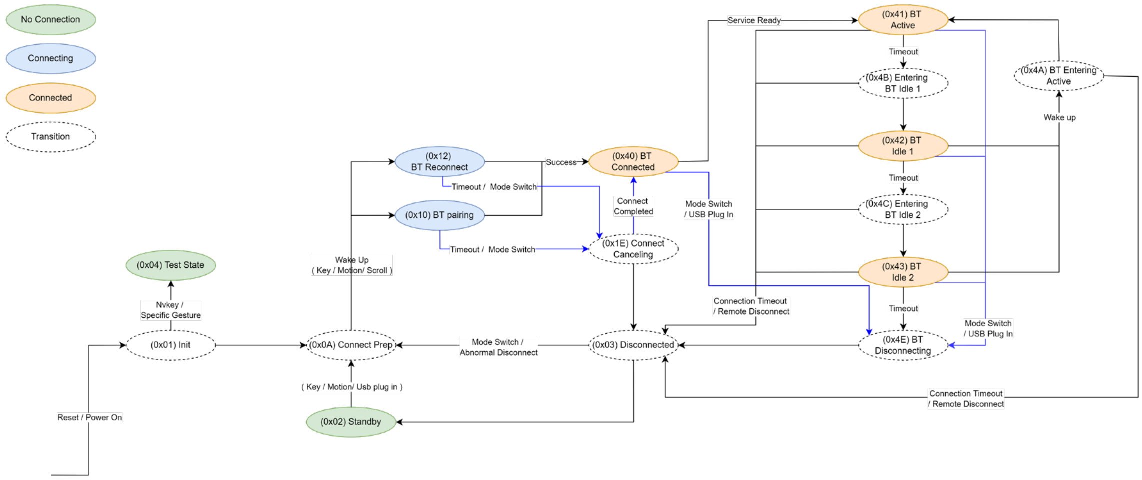

Figure 3. State Flow of Standard BT Scenario

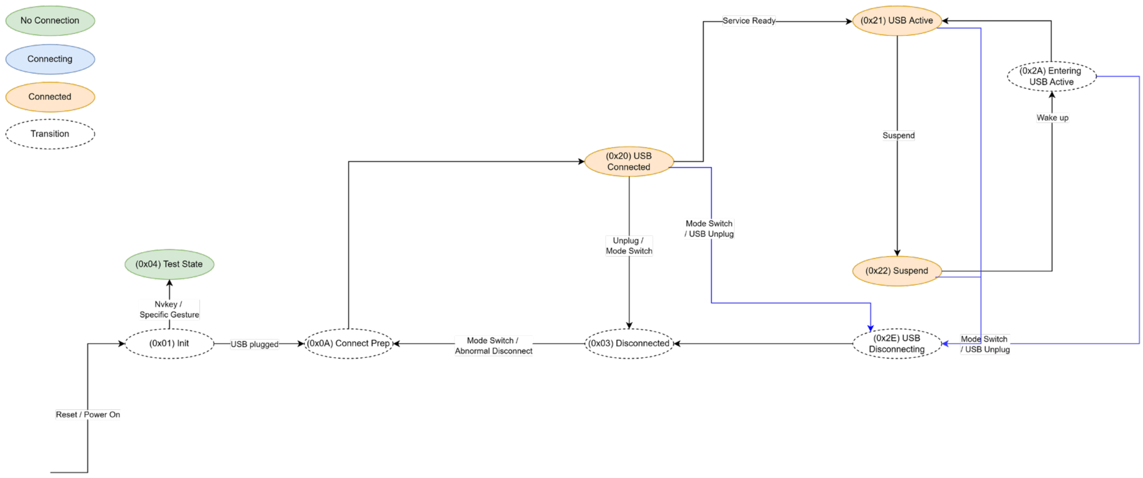

Figure 4. State Flow of a USB Scenario

2.1.2 App Button

The App Button module is a simple component within the application. It leverages the Gesture Service to recognize different gestures performed on specific keys. For example, if the user presses the mouse's Left, Middle, and Right buttons simultaneously for three seconds, the module detects this as a combo key event. By utilizing the Gesture Service, the App Button module can easily support a variety of complex input patterns without having to implement gesture recognition logic itself. Table 3 lists the currently registered gestures and their corresponding actions.

Table 1. Key Actions Used in the App Button Module

No	Key	Gesture	Description
1	Report Rate Key	Short click	An event requesting the next report rate will be announced.
2	Report Rate Key	Double clicks	An event requesting the previous report rate will be announced. *1
3	Report Rate Key	Triple clicks	An event requesting the maximum report rate will

			be announced. *1
4	DPI key	Double-click, then press and hold the key	Set the key debounce time to zero for testing.
5	DPI key	Short click	An event for requesting the next DPI will be announced.
6	DPI key	Double click	An event for requesting the previous report rate will be announced. *1
7	DPI key	Press and hold to reach level 2	An event for requesting the next wireless scenario will be announced. For example, if the device supports only one 2.4G link and one standard BT link, the connection will switch between 2.4G and BT after the event is announced.
8	Combo key	Press and hold Left & Middle & Right keys to reach level 1. (2 seconds)	An event for requesting pairing will be announced.

*1: Not included in QC test scope.

2.1.3 App LED

The LED module is responsible for controlling the LED indicators. It uses the led_middle_ctrl service. In an application, subscribe to events of interest, such as evt_app_state or evt_app_battery, and then blink the corresponding LED indicator according to the event content.

2.1.4 App Scenario

This module is specifically designed to manage device connection-related functions. It relies on the assistance of the HID Scenario Service to handle various connection requests. The module subscribes to the evt_conn_req event and performs corresponding connection tasks based on the event content, such as initiating pairing again, reconnecting, disconnecting, or switching to a different connection device. In addition, the module also subscribes to the evt_rr_change_req event. When this event is received, the module switches the report rate according to the requirements, in order to meet the data transmission frequency needs of different application scenarios.

2.1.5 App CCNI

This module is used for communication with the RISC-V processor. It is responsible for sending messages to the RISC-V processor as well as receiving messages from the RISC-V processor.

2.2 Application Framework Events

Table 2 lists the events of the application framework that are currently in common use.

Table 2. Application Framework Events in Use

No	Event Name	Descriptions
1	evt_active_status	This event is announced when waking up or entering the idle state. An event with an active status is announced by the sensor, wheel, or key. An event with a non-active status (idle) is announced by the idle detection mechanism in the RISC-V processor.
2	evt_app_state	This event is announced to indicate app state updates.
3	evt_app_battery	This event is announced to indicate battery status updates.
4	evt_pairing_request	This event is announced to request entry into the pairing state.
5	evt_rr_change_req	This event is announced to request a change in the report rate.
6	evt_rr_status	This event is announced to indicate updates to the report rate status.
7	evt_dpi_change_req	This event is announced to request a change in the DPI.
8	evt_dpi_status	This event is announced to indicate DPI updates.
9	evt_mode_change	This event is announced to request a change in the scenario.
10	evt_slide_switch_status	This event is announced to indicate the status of the slide switch.
11	evt_factory_reset	This event is announced to request a factory reset.
12	evt_key_event	This event is announced to indicate a key event.
13	evt_conn_req	This event is announced for connection-related requests.
14	evt_scenario_status	This event is announced to indicate the scenario status.
15	evt_race_cmd_req	This event is announced to forward a RACE command.
16	evt_race_cmd_rsp	This event is announced to forward a response to a RACE command.

3 Services

3.1 Application Framework Service

This service provides a communication channel between various modules in the application layer. Each module can receive and process events from the application layer through a subscription mechanism and can also submit events to the application layer. By using this service, modules achieve a high degree of modularity, minimizing interactions and coupling between them. The basic usage of this service is as follows:

AF_EVT_SUBSCRIBE_FUN: Used to subscribe to events of interest. When an event is published, the specified callback function will be invoked directly.

AF_EVT_SUBMIT: Used to publish new events. Other modules that have subscribed to this event will receive it and execute the corresponding callback function.

AF_EVT_EXTERN: Used to declare an event creation function, which is used to publish new events. For example, create_evt_pairing_request function is declared in AF_EVT_EXTERN(evt_pairing_request).

AF_EVT_DECLARE: Used to declare an event list. The content of the event list is generated based on the actual usage of AF_EVT_SUBSCRIBE_FUN.

3.2 Gesture Service

The gesture service is a highly convenient feature for detecting key behaviors. It can significantly reduce development time in the application layer. The table below lists the gestures that are currently supported.

Table 3. Events of Gesture Service

No	Event ID	Event
1	0x00	Key release event.
2	0x11~0x19	Multi-clicks 0x11: Single click event. 0x12: Double click event. … 0x19: 9 clicks event.
3	0x21~0x23	Release event for press-and-hold to reach levels 1 to 3.
4	0x31~0x39	Release event of multi-click and hold.
5	0x80	Key pressed.
6	0x91~0x93	Press-and-hold to reach levels 1~3.
7	0xA1~0xA9	Multi-click and hold event 0xA1: Single-click and hold event. 0xA2: Double-click and hold event. … 0xA9: 9 clicks and hold event.

8	0xB0~0xB9	Combo key event 0~9 0xB0: Combo key event for the 1st registered combo key. 0xB1: Combo key event for the 2nd registered combo key. … 0xB9: Combo key event for the 10th registered combo key.

3.3 HID Scenario Service

The HID scenario service currently supports three connection scenarios: 2.4G connection, USB connection, and standard Bluetooth connection. The 2.4G connection scenario is designed for low latency and requires a compatible dongle. The USB connection scenario provides a wired connection. The standard Bluetooth connection scenario can interface with common Bluetooth devices, such as laptops, Bluetooth-enabled PCs, smartphones, tablets, and more. Among these, the 2.4G and USB connection scenarios support report rates of 125Hz, 250Hz, 500Hz, 1KHz, 2KHz, 4KHz, and 8KHz. The standard Bluetooth connection scenario currently supports a maximum report rate of 133Hz.

3.4 Sensor Manager Service

This service is responsible for handling matters related to the mouse sensor, such as the sensor's operating mode and DPI settings.

3.5 User Profiles Service

The user profile service provides four default user profile records. This service allows users to quickly switch between different configurations, such as report rate, DPI settings, key remapping settings, and more.

3.6 Customer Protocol Service

The customer protocol service provides a simple interface that allows customers to easily port their proprietary communication protocols.

4 RISC-V Application

The RISC-V application layer is mainly responsible for data sampling from peripheral devices. Taking the mouse application as an example, the RISC-V application needs to perform three tasks: (1) reading the status of mouse buttons, (2) reading the x and y data from the mouse sensor, and (3) reading data from the wheel device.

The RISC-V application must register a callback function with the HID Express Service. This callback function will be invoked four times within the SPI ISR. Each invocation allows the RISC-V application to perform corresponding tasks.

HID_EXP_1ST_CB

This callback is triggered at the very beginning of the SPI ISR execution. A typical use case is to turn on the IR LED. By the time the third callback is executed, the IR LED will have been turned on for the required duration, usually around 40us.

HID_EXP_2ND_CB

In mouse applications, this callback is used to read the sensor's x and y data as well as the wheel data. The core idea of this design is to move all tasks unrelated to mouse button processing out of the third callback, thereby optimizing button latency performance.

HID_EXP_3RD_CB_GET_KEY__TIMING_CRITICAL

In mouse applications, only mouse button-related procedures are handled in this callback. Other unnecessary operations are minimized to reduce button latency.

HID_EXP_4TH_CB_POST_PROCESS

This is the final callback. At this point, the HID report has been submitted, and any remaining tasks can be handled in this callback.

5 RISC-V HID Express Service

The HID Express Service is mainly responsible for four tasks: key remapping, queue management, report packaging, and report submission. For mouse applications, most of the work is completed within the SPI ISR, while some tasks, such as parts of key remapping, require coordination with a timer.

5.1 Key Remap

When the key remap function is enabled, the first thing HID Express does after receiving the latest sampled data from the RISC-V application is to send the data to the key remap handler. The key remap module rewrites the sampled data according to the user settings. For example, the mouse side buttons can be remapped to the keyboard volume up and volume down keys. More complex applications include macros, where a mouse side button can be mapped to a series of keyboard keys, forming a sentence such as "Hello World".

5.2 Queue

There are two types of queues: the XY queue and the key queue
The xy queue is responsible for storing and calculating sensor data. If data transmission fails, the data retrieved from the xy queue remains unchanged. Once retransmission succeeds, the xy queue accumulates all unsent data.
The key queue operates in a first-in, first-out (FIFO) manner. When retrieving data from the queue, the data is not immediately removed. Only after the data has been successfully sent, another key queue API is called to remove the oldest data from the queue.

5.3 Report Packaging

Report packaging and submission vary depending on the connection type. For 2.4G connections, the HID report is limited to less than 9 bytes. In standard Bluetooth connections, there is no restriction on the HID report length. The USB mode adopts the same approach as the 2.4G mode.

5.4 Report Submission

For report submission, in both 2.4G and standard Bluetooth connections, the HID report is delivered directly to the BT controller's memory and then waits for the hardware to send it over the air. For USB connections, the HID report needs to be sent to the CM33 processor for transmission.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to Airoha Technology Corp. and/or its affiliates (collectively “Airoha”) or its licensors and is provided solely for Your internal use with Airoha’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against Airoha or any of Airoha’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify Airoha for any loss or damages suffered by Airoha for Your unauthorized use or disclosure of this Document, in whole or in part.

Airoha and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. Airoha does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY AIROHA IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. AIROHA SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. AIROHA SHALL NOT BE RESPONSIBLE FOR ANY AIROHA DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, Airoha makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does Airoha assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating Airoha’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.